

1. 기술스택

런타임: Node.js 24.21.0

언어: TypeScript // any 타입 사용 금지

프레임워크: Express

DB: Supabase

변수명 규칙: 카멜 표기법

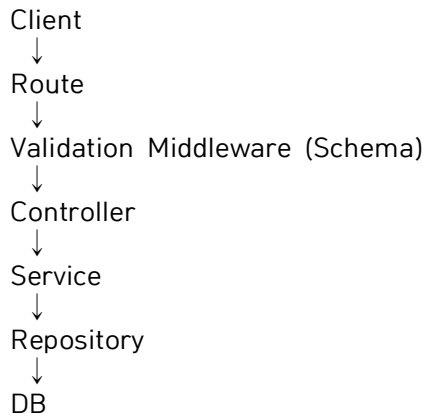
2. 프로젝트 구조

기능별 폴더

```
src/
├── config/           // 프로젝트 전반에서 사용하는 설정을 관리
├── middlewares/      // 여러 기능에서 공통으로 사용하는 미들웨어를 관리
├── modules/          // 기능별로 폴더를 분리
│   ├── auth/
│   ├── user/
│   ├── friend/
│   ├── post/
│   ├── comment/
│   ├── like/
│   ├── diary/
│   ├── photo/
│   └── notification/
├── utils/            // 프로젝트 전반에서 사용하는 공통 유틸리티를 관리
├── app.ts            // Express 애플리케이션을 구성
└── server.ts         // 서버 실행
```

각 기능 폴더 안에 Controller / Service / repository / route / schema

```
auth/
├── auth.controller.ts
├── auth.service.ts
├── auth.repository.ts
├── auth.route.ts
└── auth.schema.ts
```



Route

HTTP Method와 URL을 정의하고 Controller와 연결한다.

역할: URL 정의
HTTP Method 정의
Middleware 연결
Controller 연결

비즈니스 로직이나 DB 접근 코드는 작성하지 않는다.

Schema

외부에서 들어오는 데이터의 형식을 정의한다.

역할: 필수값 검증
타입 검증
문자열 길이 검증
숫자 범위 검증
Enum 검증
이메일 및 기타 형식 검증

Schema는 Zod를 사용한다.

실제 검증은 미들웨어에서 실행하고스 키마는 정상 데이터 형식을 정의하는 역할

Controller

HTTP 요청과 응답을 담당한다.

역할: Request 데이터 추출
Service 호출
HTTP Status Code 설정
Response 반환
Error 전달

Controller에서 직접 DB에 접근하지 않는다.

Service

서비스의 비즈니스 로직을 담당한다.

역할: 기능 수행에 필요한 조건 검사
 권한 및 접근 가능 여부 판단
 여러 Repository 호출 조합
 비즈니스 규칙 처리

DB에 직접 접근하지 않고 Repository를 사용한다.

Repository

데이터베이스 접근을 담당한다.

역할: 데이터 조회
 데이터 생성
 데이터 수정
 데이터 삭제
 Supabase Query 작성
 DB 관련 에러 처리

비즈니스 로직은 Repository에 작성하지 않는다.

미들웨어

middlewares/

—— auth.middleware.ts	// Access Token을 검증하고 인증된 사용자 정보를 설정
—— validate.middleware.ts	// Schema를 이용해 Request 데이터 검증
—— error.middleware.ts	// 공통 에러 처리
—— notFound.middleware.ts	// 존재하지 않는 API 요청 처리

인증이 필요한 API

```
Route
↓
Auth Middleware
↓
Validation Middleware
↓
Controller
```

인증이 필요하지 않은 API

```
Route
↓
Validation Middleware
↓
Controller
```

3. API 규칙

- REST API 방식을 사용한다.
- API URL은 화면이 아닌 리소스를 기준으로 설계한다.
- URL에는 동사보다 명사를 사용한다.
- 일반 리소스명은 복수형을 사용한다.
- 인증과 같이 리소스로 보기 어려운 기능은 예외적으로 단수형을 사용한다.
- URL은 소문자로 작성한다.
- 단어가 여러 개인 경우 - 을 사용한다.

api 경로 예시

```
/auth
/users
/friends
/posts
/comments
/likes
/diaries
/friend-requests
```

4. HTTP Status Code

200 OK	// 정상적인 조회/수정
201 Created	// 생성 성공
204 No Content	// 성공했지만 응답 데이터 없음
400 Bad Request	// 잘못된 요청
401 Unauthorized	// 인증 실패
403 Forbidden	// 권한 없음
404 Not Found	// 요청한 리소스 없음
409 Conflict	// 상태 충돌
500 Internal Server Error	// 서버 오류

5. Error 처리 규칙

1. 기본 원칙

- 모든 Error는 error.middleware.ts에서 공통 처리한다.
- Controller에서 직접 Error Response를 작성하지 않는다.
- Service에서는 비즈니스 로직에 따른 Error를 발생시킨다.
- Repository에서는 DB 관련 Error를 처리하거나 상위 계층으로 전달한다.
- Error Code는 UPPER_SNAKE_CASE를 사용한다.

2. Error Response 형식

```
{
  "success": false,
  "error": {
    "code": "POST_NOT_FOUND",
    "message": "게시글을 찾을 수 없습니다."
  }
}
```

3. Error 코드 예시

400 → INVALID_REQUEST

401 → AUTH_REQUIRED

403 → FORBIDDEN

404 → ?_NOT_FOUND // ?에는 못찾은 리소스 ex) USER_NOT_FOUND

409 → ALREADY_? // ?에는 충돌한 리소스명 ex) ALREADY_FRIEND

500 → INTERNAL_SERVER_ERROR

6. DB 네이밍 규칙

1. 테이블명

- 소문자 사용
- 복수형 사용
- 단어 구분은 snake_case 사용
ex) users, posts, comments, friend_requests

2. 컬럼명

- 소문자 사용
- 단어 구분은 snake_case 사용
ex) user_id, post_id, created_at, updated_at

3. Enum

- 값은 대문자로 작성한다.
- 여러 단어는 _로 구분한다.
ex) PENDING, ACCEPTED, REJECTED

4. 중간 테이블

- 다대다 관계의 경우 연결되는 두 테이블의 이름을 조합한다.
ex) post_likes, user_friends, post_tags

5. 기본 키(PK)

- 컬럼명은 id 사용
ex) id

6. 외래 키(FK)

- 참조하는 테이블의 단수형 + _id 사용
ex) user_id, post_id, comment_id, diary_id

7. Git 협업 규칙

1. 브랜치 종류

- main // 배포용, 항상 배포 가능한 상태를 유지
- develop // 개발용, 기능 개발이 완료된 작업을 통합
- feature/기능명 // 개별 기능을 개발
- release // 배포 전 최종 테스트 및 수정 작업 진행

2. 브랜치 규칙

- main에 직접 Push하지 않는다.
- develop에 직접 Push하지 않는다.
- 모든 기능 개발은 feature에서 진행한다.
- 기능 개발이 완료되면 develop으로 Merge한다.
- 배포 준비가 완료되면 develop에서 release를 생성한다.
- release에서 최종 테스트 및 배포 전 수정 작업을 진행한다.
- 테스트가 완료된 release를 main으로 Merge하여 배포한다.
- release의 변경사항은 develop에도 반영한다.

3. feature 브랜치 네이밍

feature/기능명

ex) feature/login

feature/post

feature/diary

feature/friend-request

4. 브랜치 흐름

feature/기능명



develop



release



main

5. 커밋 메세지

- 커밋 메세지 형식 = type: 설명

- type 종류

feat // 새로운 기능 추가

fix // 버그 수정

refactor // 기능 변경 없이 코드 구조 개선

docs // 문서 수정

test // 테스트 코드 작성 및 수정

chore // 설정, 패키지 등 기타 작업

6. 커밋 작성 규칙

- 커밋 메세지의 설명은 한글로 작성한다.
- Type은 소문자로 작성한다.
- : 뒤에 한 칸 띄운다.
- 커밋 메세지는 작업 내용을 간결하게 작성한다.